

AWS Setup and Security Alerting Project

Date: 04 . 05 . 2025

Prepared by: Muhydeen .O. Afolabi

Project Overview

This project involved setting up an AWS Free Tier environment to implement a real-time security alerting system focused on monitoring root user activity. The main goal was to detect and respond to potentially unauthorized use of the AWS root account by tracking the `GetCallerIdentity` API call—an action often used during reconnaissance or by attackers validating credentials. To achieve this, the project utilized key AWS services: **CloudTrail** for logging, **SNS (Simple Notification Service)** for notifications, and **EventBridge** for event-driven automation.

By integrating these tools, the system was able to generate immediate alerts via email when a sensitive action was detected, demonstrating an effective approach to enhancing cloud infrastructure security.

Objectives

- # Create an AWS Free Tier account to conduct experiments in the cloud.
- # Make sure CloudTrail is set up to record AWS API activities.
- # SNS (Simple Notification Service) from Amazon can be used to deliver email notifications.
- # Configure Amazon EventBridge so that root user actions will cause alerts to be sent.
- # To verify functionality, use the AWS CLI to test the configuration.

Technologies Used

- **AWS CloudTrail:** Tracks and logs all AWS API requests made within the account.
- **Amazon SNS:** Publishes notifications to email subscribers based on triggered events.
- **Amazon EventBridge:** Routes specific API events to SNS for alerting.
- **Kali Linux CLI:** Used to simulate API calls and test alert functionality.
- **AWS CLI:** Command Line Interface tool for interacting with AWS services programmatically

Project Steps

1. AWS Free Tier Account Setup

- Created a Free Tier AWS account on aws.amazon.com.
- Configured billing information and identity verification.
- Selected "Basic Support – Free" for cost efficiency.

The screenshot displays the AWS Management Console interface for an Amazon SNS subscription. At the top, a blue banner announces a new feature: "Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)". Below this, the subscription ID "1bc78354-b480-4f3b-ab0e-439d42329728" is prominently displayed with "Edit" and "Delete" buttons. The "Details" section is divided into two columns. The left column lists the ARN, Endpoint (midocyberworld@gmail.com), Topic (getcalleralert), and Subscription Principal. The right column shows the Status as "Confirmed" with a green checkmark and the Protocol as "EMAIL". Below the details, there are tabs for "Subscription filter policy" and "Redrive policy (dead-letter queue)". The "Subscription filter policy" tab is active, showing a message that no filter policy is configured for this subscription. The footer of the console includes the copyright notice "© 2025, Amazon Web Services, Inc. or its affiliates." and links for "Privacy", "Terms", and "Cookie preferences".

Subscription: 1bc78354-b480-4f3b-ab0e-439d42329728

Details

ARN arn:aws:sns:eu-north-1:935951869861:getcalleralert:1bc78354-b480-4f3b-ab0e-439d42329728	Status Confirmed
Endpoint midocyberworld@gmail.com	Protocol EMAIL
Topic getcalleralert	
Subscription Principal arn:aws:iam::935951869861:root	

Subscription filter policy | Redrive policy (dead-letter queue)

Subscription filter policy Info
This policy filters the messages that a subscriber receives.

No filter policy configured for this subscription.

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

2. Configured AWS CloudTrail

- Accessed **CloudTrail** in the AWS Management Console.
- Created a new trail named MyCloudTrail.
- Enabled **Management Event Logging** for all read/write events.
- Configured logs to be stored in an **S3 bucket** (my-cloudtrail-logs-uniqueid).
- Enabled **multi-region logging** for full coverage.

The screenshot shows the AWS CloudTrail console interface. At the top, there's a navigation bar with the AWS logo, a search bar containing 'cloudtrail', and various utility icons. Below the navigation bar, a green banner indicates 'Trail successfully created'. A blue banner below that provides a 'What's new' notification about strengthening data perimeters. The main section is titled 'Trails' and contains a table with columns: Name, Home region, Multi-region trail, ARN, Insights, Organization trail, S3 bucket, Log file prefix, CloudWatch Logs log group, and Status. A single trail named 'alertrail' is listed with a status of 'Logging'. To the right of the table are buttons for 'Copy events to Lake', 'Delete', and 'Create trail'.

Name	Home region	Multi-region trail	ARN	Insights	Organization trail	S3 bucket	Log file prefix	CloudWatch Logs log group	Status
alertrail	Europe (Stockholm)	Yes	arn:aws:cloudtrail:eu-north-1:935951869861:trail/alerttrail	Disabled	No	aws-cloudtrail-logs-935951869861-12f15506	-	-	Logging

- Step 1
Choose trail attributes
- Step 2
Choose log events
- Step 3
Review and create

Choose log events

Events [Info](#)

Record API activity for individual resources, or for all current and future resources in AWS account. [Additional charges apply](#)

Event type
Choose the type of events that you want to log.

☒ **Management events**
Capture management operations performed on your AWS resources.

☐ **Data events**
Log the resource operations performed on or within a resource.

☐ **Insights events**
Identify unusual activity, errors, or user behavior in your account.

☐ **Network activity events**
Network activity events provide information about resource operations performed on a resource within a virtual private cloud endpoint.

Management events [Info](#)

Management events show information about management operations performed on resources in your AWS account.

No additional charges apply to log management events on this trail because this is your first copy of management events.

API activity

3. Set Up SNS for Email Alerts

- Created an **SNS topic** named APICallAlert.
- Subscribed an email address to receive notifications.
- Confirmed the email subscription via the AWS verification link.
- Linked **CloudTrail** to SNS to trigger notifications when specific events occur.

 **New Feature**
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Application Integration

Amazon Simple Notification Service

Pub/sub messaging for microservices and serverless applications.

Amazon SNS is a highly available, durable, secure, fully managed pub/sub messaging service that enables you to decouple microservices, distributed systems, and event-driven serverless applications. Amazon SNS provides topics for high-throughput, push-based, many-to-many messaging.

Create topic

Topic name

A topic is a message channel. When you publish a message to a topic, it fans out the message to all subscribed endpoints.

[Next step](#)[Start with an overview](#)

Pricing

Amazon SNS has no upfront costs. You pay based on the number of messages you publish, the number of messages you deliver, and any additional API calls for managing topics and

Benefits and features

4. Configuring EventBridge for Root API Call Alerts

- Created an **EventBridge Rule** named GetCallerIdentityAlert.
- Defined a **custom event pattern** to detect GetCallerIdentity API calls from the root user:

```
{
  "source": ["aws.sts"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventSource": ["sts.amazonaws.com"],
    "eventName": ["GetCallerIdentity"],
    "userIdentity": {
      "type": ["Root"]
    }
  }
}
```

- Configured **SNS** as the target for this rule to send notifications.
- Allowed EventBridge to create an IAM role for permissions.

aws

Search

[Alt+S]

Global

mido

IAM

Security credentials

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management New

▼ Access reports

Access Analyzer

External access

Unused access

Analyzer settings

Credential report

Organization activity

My security credentials Root user Info

The root user has access to all AWS resources in this account, and we recommend following [best practices](#). To learn more about the types of AWS credentials and how they're used, see [AWS Security Credentials](#) in AWS General Reference

Account details

Account name

mido

Email address

mido@cyberworld@gmail.com

AWS account ID

935951869861

Canonical user ID

0e10332c80ad3d0119f00ee337354b4cdd067ef72e08ecf10ee5d0057bc7f1d2

Actions

Multi-factor authentication (MFA) (1)

Remove

Resync

Assign MFA device

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

Type	Identifier	Certifications	Created on
☐ Virtual	arn:aws:iam::935951869861:mfa/midoiphone	Not Applicable	Wed Apr 30 2025

Access keys (1)

Actions

Create access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

5. Testing the Setup via kali linux CLI

- Installed and configured **kali linux** with root credentials.
- Ran the following command to simulate an API call:

aws sts get-caller-identity

- Verified that the API call was logged in **CloudTrail**.
- Checked the SNS email inbox for the alert notification.

```
(kali㉿kali)-[~]  
$ aws sts get-caller-identity  
{  
  "UserId": "935951869861",  
  "Account": "935951869861",  
  "Arn": "arn:aws:iam::935951869861:root"  
}
```

Results & Key Takeaways

- The system successfully detected and alerted on high-privilege root account API usage in near real-time.
 - Verified the integration of **CloudTrail**, **SNS**, and **EventBridge** for a responsive and automated security monitoring setup.
 - Demonstrated how AWS native tools can be combined to build a lightweight but effective **intrusion detection mechanism**.
 - Gained practical experience in event-driven AWS architecture and real-world security best practices.
 - Ensured alerts were functional, timely, and clearly communicated via email, providing a foundation for more advanced security workflows in future deployments.
-

Conclusion

This project illustrates a practical approach to implementing proactive security monitoring within AWS. By leveraging core AWS services in an integrated fashion, it's possible to create a cost-effective and efficient alerting system that identifies potentially malicious activity, such as unauthorized access via the root account. The skills demonstrated here are directly applicable to securing real-world cloud environments, especially for small to mid-size deployments prioritizing visibility and control.